

Apunte de Cátedra – Computación II

Clase 4: Análisis Asintótico

Unidades 2 y 3: Notaciones, Crecimiento de Funciones y Escalabilidad

Año 2025 – Modalidad Híbrida

Prof. Marcelo Fabio Roldán

Departamento DACEFyN – Sede Capital – UNLaR

1. ¿Por qué el Análisis Asintótico es Fundamental?

Imagina que desarrollas una aplicación que funciona perfectamente con 100 usuarios. Pero cuando el número crece a 100,000, el sistema se "cuelga". ¿Por qué?

La respuesta está en el análisis asintótico, la herramienta matemática que nos permite predecir el comportamiento de un algoritmo a medida que el tamaño de los datos crece.

🔍 Pregunta disparadora:

"¿Por qué el algoritmo de ordenamiento burbuja funciona bien con 10 elementos, pero es inutilizable con 1 millón?"

- Con 10 elementos: tarda ~1 milisegundo.
- Con 1 millón de elementos: tarda ~11 días.

Esto no es un problema de hardware, sino de complejidad algorítmica.

¿Qué es el Análisis Asintótico?

El análisis asintótico estudia el comportamiento de un algoritmo cuando el tamaño de entrada (n) tiende al infinito. Nos permite:

- Comparar algoritmos independientemente de la máquina o el lenguaje.
- Predecir si un sistema escalará a millones de usuarios.
- Tomar decisiones de diseño informadas: *"¿Usamos búsqueda lineal o binaria?"*
- Evitar errores costosos en producción.

💡 *"El análisis asintótico es como tener una bola de cristal: te permite ver el futuro de tu software antes de escribir una sola línea de código."*

¿Por qué "Asintótico"?

La palabra proviene de las matemáticas: una asíntota es una línea a la que una curva se acerca infinitamente, sin tocarla.

En algoritmos, nos interesa la tendencia a largo plazo, no el comportamiento con datos pequeños. Por ejemplo:

- Un algoritmo A tarda 100 ms con 100 elementos.
- Un algoritmo B tarda 80 ms con 100 elementos.

¿Cuál es mejor? No podemos decidir con solo eso. Pero si sabemos que:

- A es $O(n)$ → con 1 millón de elementos: ~1 segundo.
- B es $O(n^2)$ → con 1 millón de elementos: ~11 días.

La elección es clara.

2. Las Tres Notaciones Asintóticas

Para clasificar el rendimiento de un algoritmo, usamos tres notaciones fundamentales:

Notación	Nombre	Significado	Analogía
O (Big O)	Cota Superior	"En el peor caso, el algoritmo no será peor que..."	Límite máximo de velocidad: "no excederás 120 km/h"
Ω (Omega)	Cota Inferior	"En el mejor caso, el algoritmo necesitará al menos..."	Límite mínimo: "no irás a menos de 80 km/h en autopista"
Θ (Theta)	Cota Ajustada	"El algoritmo se comporta exactamente como..."	Velocidad promedio: "tu velocidad será 100 km/h"

✦ Regla clave:

- Si $T(n)=O(n^2)$, el algoritmo es a lo sumo cuadrático.
 - Si $T(n)=\Omega(n)$, el algoritmo es al menos lineal.
 - Si $T(n)=\Theta(n \log n)$, el algoritmo es exactamente $n \log n$.
-

Definiciones Formales (Simplificadas)

Sea $f(n)$ el tiempo de ejecución de un algoritmo.

- $f(n)=O(g(n))$ si existe una constante c y un n_0 tal que $f(n)\leq c\cdot g(n)$ para todo $n>n_0$.
- $f(n)=\Omega(g(n))$ si existe una constante c y un n_0 tal que $f(n)\geq c\cdot g(n)$ para todo $n>n_0$.
- $f(n)=\Theta(g(n))$ si $f(n)=O(g(n))$ y $f(n)=\Omega(g(n))$.

📌 Ejemplo:

Para $f(n)=3n^2+5n+2$:

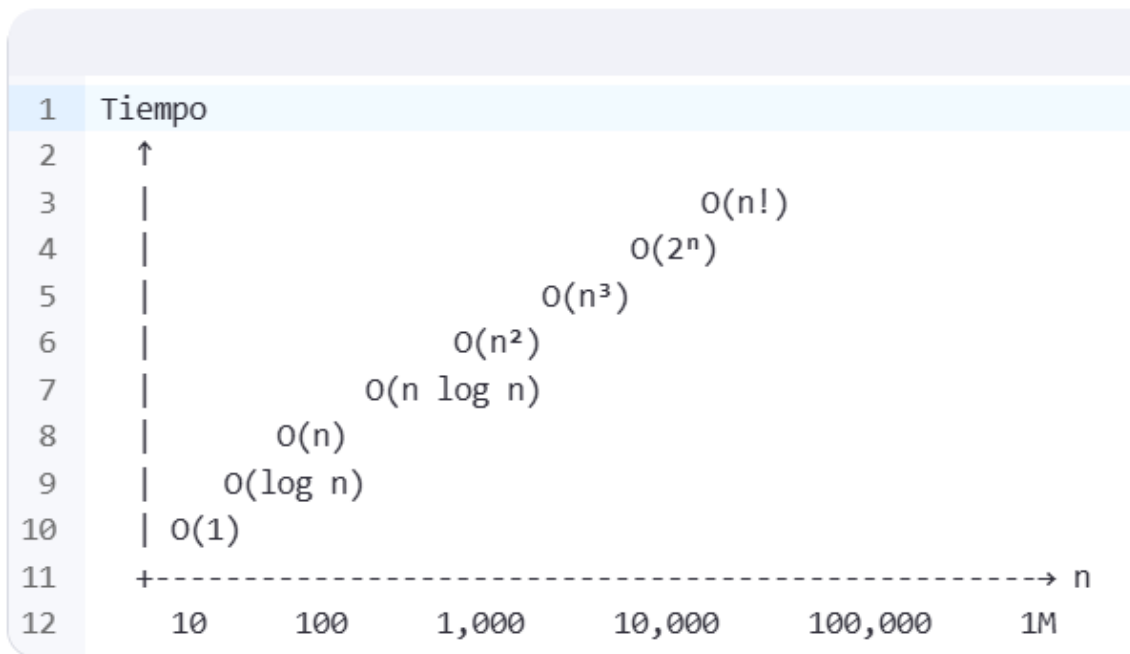
- $f(n)=O(n^2)$
- $f(n)=\Omega(n^2)$
- $f(n)=\Theta(n^2)$

3. Crecimiento de Funciones: La Jerarquía de la Eficiencia

El crecimiento de funciones determina la escalabilidad de un algoritmo. Aquí está el orden de eficiencia (de mejor a peor):

Función	Nombre	Ejemplo de algoritmo
$O(1)$	Constante	Acceso a array por índice, hashing (YouTube)
$O(\log n)$	Logarítmica	Búsqueda binaria, árboles de búsqueda (MySQL)
$O(n)$	Lineal	Búsqueda lineal, recorrer una lista (Facebook timeline)
$O(n\log n)$	Linealítmica	Merge Sort, Heap Sort (Google, Netflix)
$O(n^2)$	Cuadrática	Bubble Sort, selección, matrices anidadas
$O(n^3)$	Cúbica	Multiplicación de matrices (algoritmo naívo)
$O(2^n)$	Exponencial	Backtracking sin poda, fuerza bruta (N-Reinas, subset sum)
$O(n!)$	Factorial	Problema del viajante (TSP)

Gráfico Comparativo del Crecimiento



🔥 Dato impactante:

Con 1 millón de elementos:

- $O(1)$: 1 microsegundo
- $O(\log n)$: 20 microsegundos
- $O(n)$: 1 segundo
- $O(n \log n)$: 20 segundos
- $O(n^2)$: 11 días
- $O(2^n)$: Más de edad del universo

💡 "Un algoritmo $O(n^2)$ puede ser aceptable para 1,000 elementos, pero es inviable para 1 millón. La elección del algoritmo determina si tu startup escala o quiebra."

4. Casos Prácticos: Mejor, Promedio y Peor Caso

Un algoritmo puede tener diferentes comportamientos según la entrada.

Ejemplo: Búsqueda Lineal en un Array

python

```
1 def buscar(arr, x):
2     for i in range(len(arr)):
3         if arr[i] == x:
4             return i
5     return -1
```

Caso	Descripción	Complejidad
Mejor caso	El elemento está en la primera posición.	$O(1)$
Peor caso	El elemento está al final o no está.	$O(n)$
Caso promedio	El elemento está en una posición aleatoria.	$O(n)$

✦ Importante: En análisis asintótico, nos enfocamos en el peor caso porque garantiza un rendimiento máximo.

Ejemplo: Ordenamiento por Inserción

python

```
1 def insertion_sort(arr):
2     for i in range(1, len(arr)):
3         key = arr[i]
4         j = i - 1
5         while j >= 0 and arr[j] > key:
6             arr[j + 1] = arr[j]
7             j -= 1
8         arr[j + 1] = key
```

Caso	Descripción	Complejidad
Mejor caso	Array ya ordenado.	$O(n)$
Peor caso	Array ordenado en orden inverso.	$O(n^2)$

Caso	Descripción	Complejidad
Caso promedio	Permutación aleatoria.	$O(n^2)$

❑ Conclusión: Aunque en el mejor caso es lineal, su complejidad dominante es $O(n^2)$, lo que lo hace inadecuado para grandes datasets.

5. Aplicaciones Reales en la Industria

El análisis asintótico no es teoría: es la base del software que usamos todos los días.

Empresa	Algoritmo	Complejidad	Impacto
Google Search	Búsqueda en índices invertidos	$O(\log n)$	Encuentra resultados entre billones de páginas en milisegundos
Facebook	Carga de timeline	$O(n)$	Muestra posts de miles de amigos en tiempo real
Netflix	Recomendaciones	$O(n \log n)$	Filtra y ordena contenido para 250+ millones de usuarios
Amazon	Procesamiento de transacciones	$O(n)$ a $O(n \log n)$	Maneja millones de pedidos diarios
YouTube	Acceso a videos	$O(1)$ (hashing)	Permite acceso instantáneo a miles de millones de videos

🌐 Caso real: En 2023, Amazon optimizó un algoritmo de $O(n^2)$ a $O(n \log n)$, ahorrando \$120 millones USD/año en costos de cómputo.

6. Reglas Prácticas para Calcular la Complejidad

a) Loops Simples

python

```
1 v for i in range(n):  
2     # operación O(1)
```

→ $O(n)$

b) Loops Anidados

python

```
1 v for i in range(n):  
2 v     for j in range(n):  
3         # operación O(1)
```

→ $O(n^2)$

c) Loops con División

python

```
1 i = n  
2 v while i > 1:  
3     i = i // 2
```

→ $O(\log n)$

d) Combinaciones

python

```
1 v for i in range(n):           # O(n)  
2 v     for j in range(log n):  # O(log n)  
3         # operación O(1)
```

→ $O(n \log n)$

e) Recursión

python

```
1 def fib(n):  
2     if n <= 1:  
3         return n  
4     return fib(n-1) + fib(n-2)
```

→ $O(2^n)$ (sin memoización)

7. Errores Comunes en el Análisis Asintótico

Error	Corrección
❑ Incluir constantes: $O(5n)$	❑ Ignorar constantes: $O(n)$
❑ Incluir términos menores: $O(n^2+n)$	❑ $O(n^2)$
❑ Confundir Big O con tiempo exacto	❑ Big O es una cota, no un valor exacto
❑ Analizar solo el mejor caso	❑ Enfocarse en el peor caso
❑ Ignorar el costo de operaciones internas	❑ Considerar acceso a memoria, llamadas a funciones

💡 Consejo:

- Ignora constantes: $O(100n)=O(n)$
- Ignora términos menores: $O(n^2+n)=O(n^2)$
- Multiplica en loops anidados: $O(n) \times O(m)=O(nm)$

8. Laboratorio Práctico: Comparación Empírica

Objetivo

Medir y graficar el tiempo de ejecución de diferentes algoritmos con datasets crecientes.

Tareas

1. Implementar:

- Búsqueda lineal ($O(n)$)
- Búsqueda binaria ($O(\log n)$)
- Ordenamiento burbuja ($O(n^2)$)
- Merge Sort ($O(n \log n)$)

2. Medir:

- Tiempo de ejecución (usar time en Python)
- Número de comparaciones
- Uso de memoria (memory_profiler)

3. Usar datasets de tamaño: 1K, 10K, 100K, 1M

4. Graficar los resultados y comparar con la teoría.

Herramientas Recomendadas

- Python: time, cProfile, line_profiler
- Haskell: criterion
- Visualización: matplotlib, gnuplot

📌 Consejo: Usa el "threshold de 64 elementos": para arrays pequeños, la búsqueda lineal puede ser más rápida que la binaria por mejor localidad de cache.

9. Conceptos Clave Aprendidos

- 📖 El análisis asintótico predice el comportamiento de un algoritmo con datos masivos.
- 📖 Las notaciones O , Ω , Θ permiten clasificar algoritmos por su eficiencia.
- 📖 El crecimiento de funciones determina la escalabilidad:
 $O(1) < O(\log n) < O(n) < O(n \log n) < O(n^2) < O(2^n)$.
- 📖 En la práctica, nos enfocamos en el peor caso para garantizar rendimiento.
- 📖 Las grandes empresas (Google, Netflix, Amazon) dependen de algoritmos eficientes para escalar.
- 📖 Un algoritmo $O(n^2)$ puede colapsar con datos grandes, aunque funcione bien con datos pequeños.

💬 "El análisis asintótico no es un tema para aprobar un examen; es una herramienta que usarás todos los días como profesional. La diferencia entre un puente que soporta 10 autos y uno que soporta 10,000."

10. Bibliografía y Recursos Recomendados

Bibliografía Básica

- Paniagua Arís, E. (2003). *Lógica computacional*. Thomson.
- Jiménez Murillo, J. A. (2008). *Matemáticas para Computación*. Alfaomega.

Bibliografía Complementaria

- Cormen, T. H. et al. (2022). *Introduction to Algorithms* (4ª ed.). MIT Press.
→ Capítulo 3: Growth of Functions.
- García Muñoz, A. (2010). *Matemática discreta para la computación*. Universidad de Jaén.

Recursos en Línea

- VisuAlgo – Asymptotic Notation Visualizer
⇒ <https://visualgo.net/en/sorting>
- Big O Cheat Sheet
⇒ <https://www.bigocheatsheet.com/>
- Haskell MOOC – Performance Analysis
⇒ <https://haskellmooc.co.uk/tutorial1>
- GitHub – Algorithm Complexity Examples
⇒ <https://github.com/shekhargulati/99-problems/tree/master/haskell>